Case Study 4: ATM Transaction Sequence Simulation

Activity sheet case study #13 | Concepts: Tuples, tuple unpacking, conditional statements, loops, variables (state), operators (modulus)

1. Problem Statement

Simulate a bank account that receives a sequence of deposit and withdrawal requests. The program must keep a running balance, enforce withdrawal limits, reject invalid transactions with a reason and print a summary of all accepted and rejected transactions.

2. Objectives

- Process a list of (type, amount) tuples in order using a loop.
- Validate each request with layered conditional checks.
- Maintain state (balance and daily withdrawals) across iterations.
- Report a clear per-transaction status and a final summary.

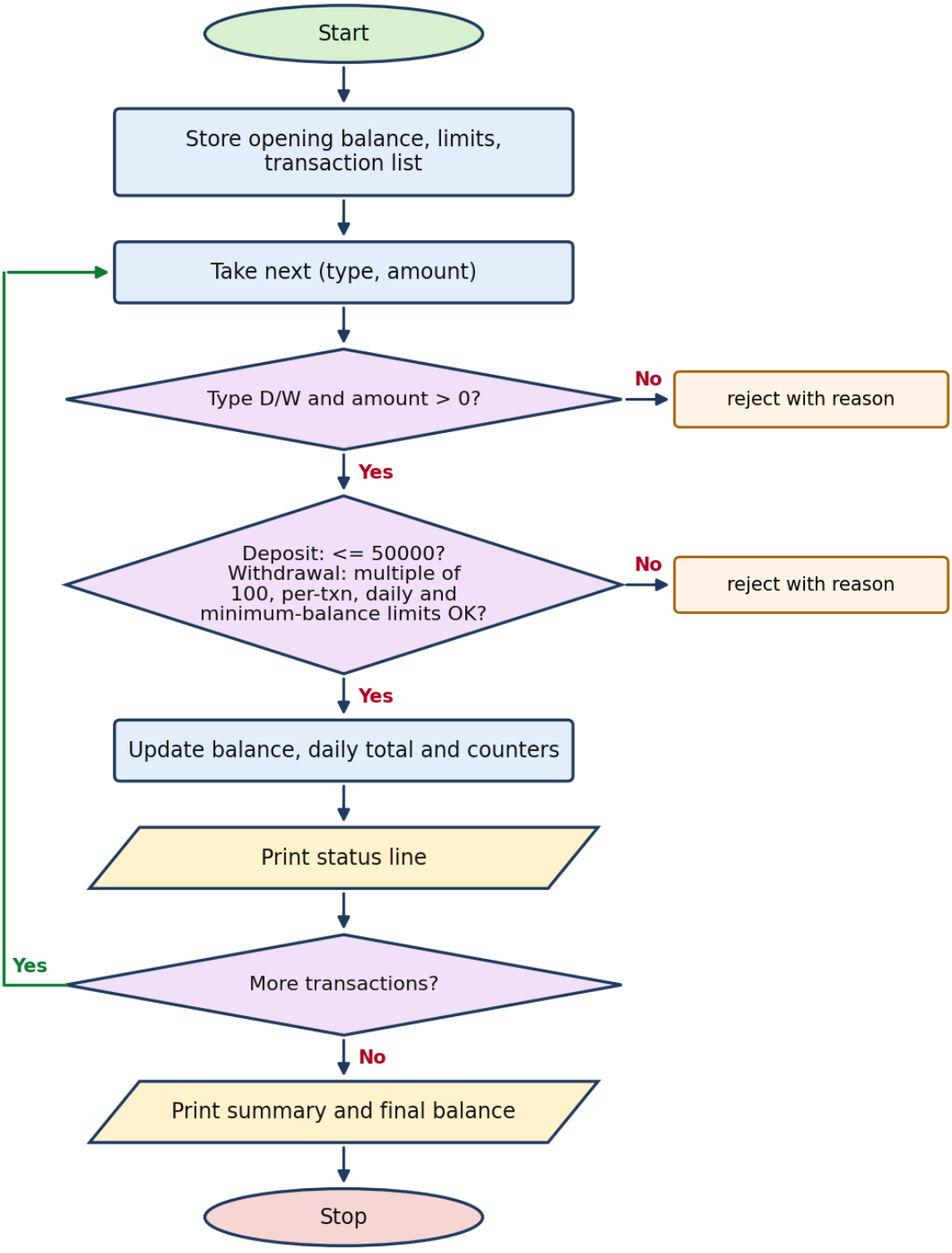
3. Assumptions

- Transaction types: 'D' for deposit and 'W' for withdrawal; any other letter is invalid.
- Amounts must be positive; a deposit may not exceed Rs 50000.
- Withdrawals must be multiples of 100, at most Rs 10000 each and Rs 20000 in total for the day.
- A minimum balance of Rs 500 must remain after every withdrawal.
- The sequence represents one day of activity; the opening balance is Rs 10000.

4. Algorithm

1. Store the opening balance, limits and the transaction list; set counters to zero.
2. For each transaction, unpack the type and amount.
3. Check in order: valid type, positive amount, then the rules specific to deposits or withdrawals.
4. If any check fails, store the reason and print REJECTED with the reason; the balance is unchanged.
5. Otherwise update the balance (and the daily withdrawal total) and the counters, then print ACCEPTED with the new balance.
6. After the loop print the number and total of deposits and withdrawals, rejected count and final balance.

5. Flowchart



6. Pseudocode

```
START
  balance <- opening ; withdrawn_today <- 0 ; counters <- 0
  FOR each (kind, amount) in transactions
      reason <- ""
      IF kind is not D and not W THEN reason <- "unknown type"
      ELSE IF amount <= 0 THEN reason <- "amount must be positive"
      ELSE IF kind = D THEN
```

```
            IF amount > MAX_DEPOSIT THEN reason <- "deposit above limit"
        ELSE
            IF amount MOD 100 != 0 THEN reason <- "not multiple of 100"
            ELSE IF amount > PER_LIMIT THEN reason <- "per-withdrawal limit"
            ELSE IF withdrawn_today + amount > DAILY_LIMIT THEN reason <- "daily limit"
            ELSE IF balance - amount < MIN_BALANCE THEN reason <- "insufficient balance"
        IF reason = "" THEN apply the transaction ; PRINT ACCEPTED
        ELSE PRINT REJECTED with reason
    END FOR
    PRINT summary
STOP
```

7. Python Source Code

```
# Case Study 13: ATM Transaction Sequence Simulation
# ---- INPUT DATA ----
opening_balance = 10000
transactions = [("D", 5000), ("W", 2500), ("W", 12000), ("W", 150),
                ("D", -300), ("W", 9000), ("X", 100), ("W", 8000),
                ("W", 3000)]
# ---- END DATA ----

MIN_BALANCE = 500
PER_WITHDRAWAL_LIMIT = 10000
DAILY_WITHDRAWAL_LIMIT = 20000
MAX_DEPOSIT = 50000

balance = opening_balance
withdrawn_today = 0
deposits = 0
deposit_total = 0
withdrawals = 0
withdrawal_total = 0
rejected = 0

print("Opening balance:", opening_balance)
print("-" * 66)
if len(transactions) == 0:
    print("No transactions to process.")

number = 0
for txn in transactions:
    kind, amount = txn                    # tuple unpacking
    number = number + 1
    if kind == "D":
        name = "Deposit"
    elif kind == "W":
        name = "Withdraw"
    else:
        name = "Unknown"

    reason = ""
    if kind != "D" and kind != "W":
        reason = "unknown transaction type"
    elif amount <= 0:
        reason = "amount must be positive"
    elif kind == "D":
        if amount > MAX_DEPOSIT:
            reason = "deposit above limit"
    else:
        if amount % 100 != 0:
            reason = "not a multiple of 100"
        elif amount > PER_WITHDRAWAL_LIMIT:
            reason = "above per-withdrawal limit"
        elif withdrawn_today + amount > DAILY_WITHDRAWAL_LIMIT:
            reason = "daily limit exceeded"
        elif balance - amount < MIN_BALANCE:
            reason = "insufficient balance"

    if reason == "":
        if kind == "D":
            balance = balance + amount
            deposits = deposits + 1
            deposit_total = deposit_total + amount
        else:
            balance = balance - amount
            withdrawn_today = withdrawn_today + amount
            withdrawals = withdrawals + 1
            withdrawal_total = withdrawal_total + amount
        print("{:>2}. {:<9}{:>7}  ACCEPTED   balance = {}".format(
              number, name, amount, balance))
    else:
        rejected = rejected + 1
        print("{:>2}. {:<9}{:>7}  REJECTED   {}".format(
              number, name, amount, reason))

print("-" * 66)
print("TRANSACTION SUMMARY")
print("Deposits accepted   :", deposits, "totalling", deposit_total)
print("Withdrawals accepted:", withdrawals, "totalling", withdrawal_total)
print("Transactions rejected:", rejected)
print("Final balance:", balance)
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Default mixed sequence	9 transactions	Final balance 500, 5 rejections	`Final balance: 500 | Transactions rejected: 5`	**PASS**
2	Per-withdrawal limit	W 12000	Rejected	`1. Withdraw 12000 REJECTED above per-withdrawal limit`	**PASS**
3	Multiple-of-100 rule	W 150	Rejected	`1. Withdraw 150 REJECTED not a multiple of 100`	**PASS**
4	Daily limit (opening 50000)	W 10000, 10000, 100	Third withdrawal rejected	`3. Withdraw 100 REJECTED daily limit exceeded | Final balance: 30000`	**PASS**
5	Minimum balance kept	W 9600 from 10000	Balance unchanged	`1. Withdraw 9600 REJECTED insufficient balance | Final balance: 10000`	**PASS**
6	Deposit above limit	D 60000	Rejected	`1. Deposit 60000 REJECTED deposit above limit`	**PASS**
7	Empty sequence	[]	Balance stays 10000	`No transactions to process. | Final balance: 10000`	**PASS**

9. Output Screenshot

Terminal output of `python case04.py` using the default data:



10. Conclusion

The simulation keeps account state across a sequence of operations, applies every limit in a fixed order and always explains why a transaction was rejected. The layered if-elif checks make the rules easy to extend and the summary confirms that the final balance equals the opening balance plus accepted deposits minus accepted withdrawals.